CLASIFICACIÓN DE ALGORITMOS DE BÚSQUEDA

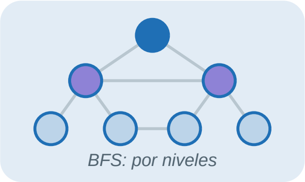
¿Qué es un algoritmo de búsqueda?

Es un **procedimiento sistemático** que localiza un elemento, una solución o un camino dentro de un conjunto de datos o de un espacio de estados. Se evalúa con cuatro criterios que se usarán en toda esta infografía.

- **Tiempo** (complejidad)
- **Memoria** (espacio)
- **Completitud** (si halla solución)
- **Optimalidad** (si halla la mejor)

1 Según la información del dominio

BÚSQUEDA NO INFORMADA (CIEGA)

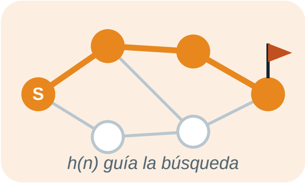

BFS: por niveles

No usa estimaciones: explora a ciegas, basándose solo en las reglas del problema.

Algoritmos

Algoritmo	Descripción
BFS (Anchura)	Expande por niveles con una cola FIFO.
DFS (Profundidad)	Baja por una rama hasta el fondo; usa una pila.
UCS (Costo uniforme)	Expande el nodo de menor costo acumulado g(n).
DLS (Prof. limitada)	DFS con un límite de profundidad l.
IDS (Prof. iterativa)	Repite DLS aumentando el límite: 0, 1, 2, 3...

BÚSQUEDA INFORMADA (HEURÍSTICA)



h(n) guía la búsqueda

Usa conocimiento o estimaciones heurísticas h(n) para priorizar el camino más prometedor.

Algoritmos

Algoritmo	Descripción
Greedy Best-First	Elige el menor h(n): rápida, pero no óptima.
A* (f = g + h)	Minimiza f(n) = g(n) + h(n); óptima con h admisible.
IDA* (A* iterativo)	A* con umbral de f creciente; poca memoria.
SMA* (memoria acotada)	Descarta el peor nodo cuando la memoria se llena.
Beam Search (haz)	Conserva solo los k mejores nodos de cada nivel.

2 Según el objetivo de la búsqueda

Búsqueda de camino

Encuentra la secuencia de acciones o la ruta de un estado inicial a una meta.

Algoritmos:
BFS, DFS, UCS, A*, IDA*

Ejemplo: Rutas en un mapa, rompecabezas de 8 piezas

Búsqueda de óptimo (optimización)

Maximiza o minimiza una función objetivo; importa el estado final, no el camino.

Algoritmos:
Hill Climbing, Simulated Annealing (recocido simulado), Búsqueda Tabú, Algoritmos Genéticos

Ejemplo: Horarios, ajuste de parámetros

Búsqueda con adversario

Decide la mejor jugada cuando otro agente busca lo contrario.

Algoritmos:
Minimax, Poda Alfa-Beta, MCTS

Ejemplo: Ajedrez, damas, Go

Búsqueda con restricciones (CSP)

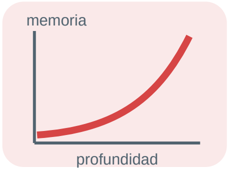
Asigna valores a variables de modo que se cumplan todas las reglas.

Algoritmos:
Backtracking, Forward Checking, AC-3

Ejemplo: Sudoku, coloreado de mapas

3 Según el uso de memoria

Memoria exponencial

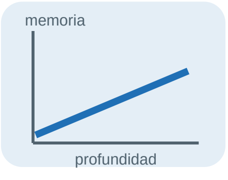


$O(b^d)$

Guarda todos los nodos generados; la memoria crece exponencialmente con la profundidad.

Algoritmos: **BFS, UCS, A*, Greedy**

Memoria lineal

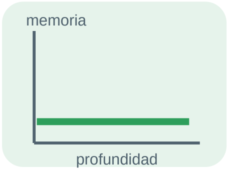


$O(b \cdot m)$

Solo guarda la rama actual y los hermanos pendientes: usa muy poca memoria.

Algoritmos: **DFS, DLS, IDS, IDA***

Memoria acotada o O(1)



O(1) o fija

Configurable o casi nula: nunca supera un tope definido de antemano.

Algoritmos: **SMA*, Beam Search, Hill Climbing**

4 Según la garantía de optimalidad

Óptimos

Garantizan siempre la mejor solución (menor costo).

Algoritmo	Condición
BFS	costos de paso iguales
UCS	costos positivos
IDS	costos de paso iguales
A*	con heurística admisible
IDA*	con heurística admisible
SMA*	si la solución cabe en memoria
Minimax + poda α-β	juegos de suma cero

No óptimos

Respuestas rápidas, pero no aseguran el menor costo.

Algoritmo	Motivo
DFS	devuelve la primera solución que halla
DLS	el límite puede cortar la mejor
Greedy Best-First	ignora el costo recorrido g(n)
Hill Climbing	se atasca en óptimos locales
Beam Search	descarta ramas que podrían ser mejores
Algoritmos genéticos	heurísticos, sin garantía

Completo = si existe una solución, el algoritmo siempre la encuentra. **Óptimo** = la solución que devuelve es la de menor costo. **Heurística admisible** = nunca sobreestima el costo real hasta la meta: $h(n) \le h^*(n)$. Un algoritmo puede ser completo y no óptimo (DFS en espacios finitos, Greedy con control de repetidos).

5 Comparativa de propiedades

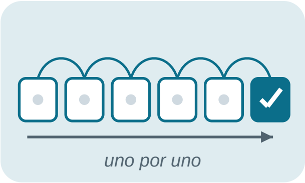
Algoritmo	Tipo	¿Completo?	¿Óptimo?	Tiempo	Espacio
● **BFS**	Ciega	✓ b finito	✓ costos iguales	$O(b^d)$	$O(b^d)$
● **DFS**	Ciega	✗ ramas infinitas	✗	$O(b^m)$	$O(b \cdot m)$
● **UCS**	Ciega	✓ costos > 0	✓	$O(b^{C^*/\varepsilon})$	$O(b^{C^*/\varepsilon})$
● **DLS**	Ciega	✗ si l < d	✗	$O(b^l)$	$O(b \cdot l)$
● **IDS**	Ciega	✓	✓ costos iguales	$O(b^d)$	$O(b \cdot d)$
● **Greedy Best-First**	Informada	✗ puede ciclar	✗	$O(b^m)$	$O(b^m)$
● **A***	Informada	✓	✓ h admisible	$O(b^d)$ *	$O(b^d)$
● **IDA***	Informada	✓	✓ h admisible	$O(b^d)$ *	$O(b \cdot d)$
● **SMA***	Informada	✓ si cabe	✓ si cabe	depende de h	acotado
● **Beam Search**	Informada	✗	✗	$O(k \cdot b \cdot d)$	$O(k \cdot b)$
● **Hill Climbing**	Local	✗ óptimos locales	✗	O(iteraciones)	O(1)

b = factor de ramificación · **d** = profundidad de la solución más superficial · **m** = profundidad máxima · **l** = límite de profundidad · **k** = ancho del haz · **C*** = costo de la solución óptima · **ε** = costo mínimo de una acción. * En la práctica depende de la calidad de la heurística.

6 Búsqueda clásica en colecciones de datos

Secuencial

Lineal · con centinela

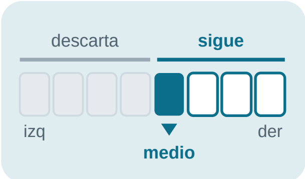


O(n) tiempo promedio

Recorre uno por uno. No requiere orden.

Listas ordenadas

Binaria · interpolación · saltos · exponencial

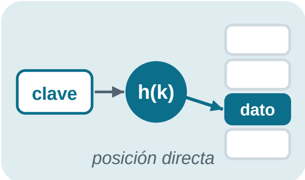


O(log n) tiempo promedio

Descarta mitades. Requiere datos ordenados.

Transformación de clave

Tabla hash

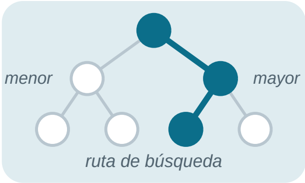


O(1) tiempo promedio

Calcula la posición con una función hash.

Árboles de búsqueda

ABB · AVL · rojo-negro

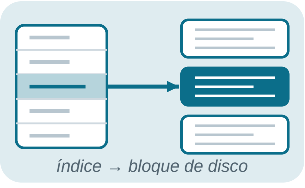


O(log n) tiempo promedio

Elige hijo izquierdo o derecho en cada nodo.

Búsqueda externa

Sec. indexada · árbol B · árbol B+ · hash externo



O(log n) tiempo promedio

Un índice lleva al bloque de disco correcto.

7 Guía rápida de selección: ¿cuál usar?

Problemas de inteligencia artificial (espacios de estados)

- ¿Sin heurística y con costos iguales? Sí → **BFS o IDS** (*IDS si hay poca memoria*). No →
- ¿Sin heurística y con costos variables? Sí → **UCS** (*cola de prioridad por g(n)*). No →
- ¿Tienes heurística admisible? Sí → **A* o IDA*** (*IDA* si falta memoria*). No →
- ¿Hay un adversario o juego? Sí → **Minimax o MCTS** (*poda α-β para acelerar*). No →
- ¿Es para optimizar un estado? Sí → **Hill Climbing / Recocido** (*búsqueda local, memoria O(1)*). No →
- ¿Es para resolver reglas (CSP)? Sí → **Backtracking** (*con Forward Checking y AC-3*)

Colecciones de datos (búsqueda clásica)

- ¿Pocos datos o sin ordenar? Sí → **Búsqueda lineal** (*O(n)*). No →
- ¿Datos ordenados en memoria? Sí → **Búsqueda binaria** (*O(log n)*). No →
- ¿Consultas por clave exacta, muy frecuentes? Sí → **Tabla hash** (*O(1) promedio*). No →
- ¿Datos que cambian y deben seguir ordenados? Sí → **AVL / rojo-negro** (*O(log n) garantizado*). No →
- ¿Datos almacenados en disco? Sí → **Árbol B+ / hash externo** (*pocos accesos a disco*)

Clasificación basada en los textos estándar de inteligencia artificial (p. ej., Russell y Norvig). Las complejidades son teóricas.